AWS DynamoDB ile CQRS View Uygulaması

Muhammed Kayra Bulut
Öğrenci No: 25501805
Ders: İleri Yazılım Mimarisi
Dersin Yürütücüsü: Dr. Göksel BİRİCİK
8 Aralık 2025

***Özet*—Bu rapor, CQRS (Command Query Responsibility Segregation) mimarisinde AWS DynamoDB kullanarak bir view uygulamasının nasıl gerçekleştirileceğini detaylı olarak açıklamaktadır. CQRS tasarımı kullanılırken karşılaşılan çeşitli tasarım sorunları ve bunların çözüm yolları incelenecektir. Ayrıca AWS DynamoDB'nin NoSQL yapısının bu mimariye entegrasyonu ele alınacaktır.**

***Index Terms*—CQRS, AWS DynamoDB, Yazılım Mimarisi, NoSQL, Event Sourcing.**

I. Giriş

Bu rapor, CQRS (Command Query Responsibility Segregation) mimarisinde AWS DynamoDB kullanarak bir view uygulamasının nasıl gerçekleştirileceğini detaylı olarak açıklamaktadır. CQRS tasarımı kullanılırken karşılaşılan çeşitli tasarım sorunları ve bunların çözüm yolları incelenecektir.

AWS DynamoDB, Amazon bulut platformunda hizmet olarak sunulan ölçeklenebilir bir NoSQL veritabanıdır. DynamoDB veri modeli, JSON nesnelerine benzer şekilde hiyerarşik ad-değer çiftlerinin koleksiyonları olan öğeleri içeren tablolardan oluşur.

II. CQRS View Mimarisi

A. Order History Service Tasarımı

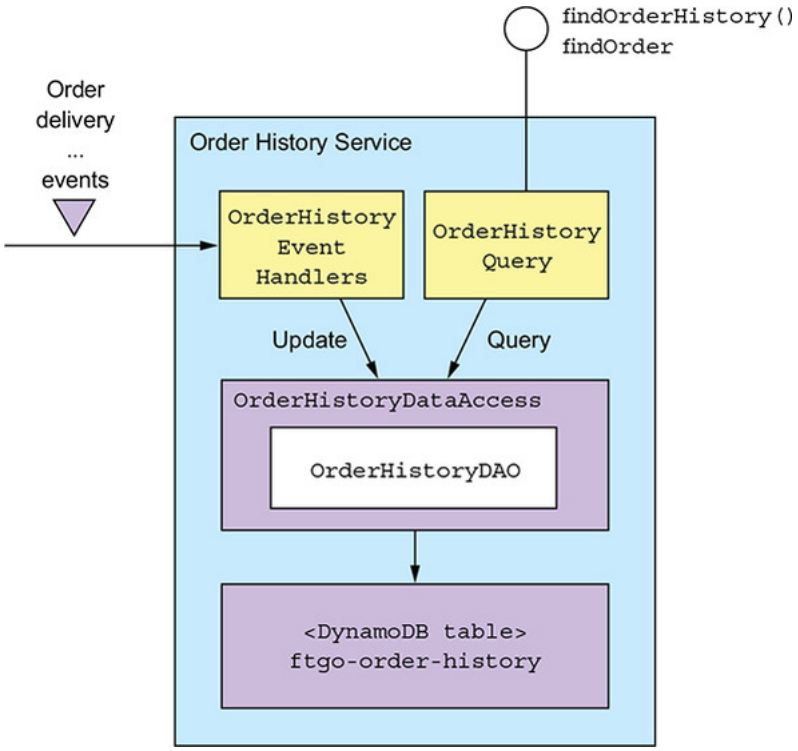
`findOrderHistory()` operasyonu için CQRS view, birden fazla servisten olayları tükettiği için bağımsız bir Order View Service olarak implemente edilmiştir. Bu servis iki temel operasyonu gerçekleştiren bir API'ye sahiptir:

- `findOrderHistory()`: Müşteri sipariş geçmişini sorgular
- `findOrder()`: Tekil sipariş bilgisi sorgular

B. Modül Yapısı

Order History Service, geliştirme ve sınamayı kolaylaştırmak için her biri belirli bir sorumluluğu yerine getiren modüllerden oluşur:

1) **OrderHistoryEventHandlers**: Çeşitli servisler tarafından yayınlanan olayları dinler ve OrderHistoryDAO'yu çağırır
2) **OrderHistoryQuery API modülü**: REST endpoint'lerini uygular
3) **OrderHistoryDataAccess**: ftgo-order-history DynamoDB tablosunu güncelleyen ve sorgulayan metodları tanımlayan OrderHistoryDAO'yu ve yardımcı sınıfları içerir
4) **ftgo-order-history DynamoDB tablosu**: Siparişleri saklayan tablo



Şekil 1. Order History Service'in tasarımı. OrderHistoryEventHandlers olaylara cevap olarak veritabanını günceller, OrderHistoryQuery modülü veritabanını sorgulayarak sorgu operasyonlarını gerçekleştirir.

III. Event Handler Modülü

A. OrderHistoryEventHandlers Yapısı

Bu modül, olayları işleyen ve DynamoDB tablosunu güncelleyen olay işleyicilerinden oluşur. Her olay işleyici, olaydan türetilen argümanlarla bir OrderHistoryDao metodunu çağıran basit metotlardır.

```
public class OrderHistoryEventHandlers {

  private OrderHistoryDao orderHistoryDao;

  public OrderHistoryEventHandlers(OrderHistoryDao
      orderHistoryDao) {
    this.orderHistoryDao = orderHistoryDao;
  }

  public void handleOrderCreated(DomainEventEnvelope<
      OrderCreated> dee) {
    orderHistoryDao.addOrder(makeOrder(dee.getAggregateId()
        , dee.getEvent()),
                             makeSourceEvent(dee));
  }

  private Order makeOrder(String orderId, OrderCreatedEvent
      event) {
    // ...
  }

  public void handleDeliveryPickedUp(
                      DomainEventEnvelope<DeliveryPickedUp>
                            dee) {
   orderHistoryDao.notePickedUp(dee.getEvent().getOrderId()
       ,
          makeSourceEvent(dee));
  }
}
```

Listing 1. Event handler'ların OrderHistoryDao'yu çağırması

Her olay işleyici, olayı ve olayı tanımlayan bazı meta verileri içeren `DomainEventEnvelope` tipinde tek bir parametreye sahiptir. Mesela, `handleOrderCreated()` metodu bir OrderCreated olayını işlemek için çağrılır ve veritabanında bir Order oluşturmak için `orderHistoryDao.addOrder()` metodunu çağırır.

IV. DynamoDB Veri Modelleme ve Sorgu Tasarımı

A. Tasarım Sıkıntıları

Birçok NoSQL veritabanı gibi DynamoDB'nin de RDBMS'lerin sağladığından çok daha az güçlü veri erişim operasyonları vardır. Bundan dolayı, verilerin nasıl saklanacağı konusu önemlidir. Bilhassa sorgular genellikle şema tasarımını belirler.

Ele alınması gereken temel tasarım sorunları şunlardır:

- ftgo-order-history tablosunun tasarlanması
- findOrderHistory sorgusu için index tanımlanması
- findOrderHistory sorgusunun implementasyonu
- Sorgu sonuçlarının sayfalandırılması
- Siparişlerin güncellenmesi
- Mükerrer olayların tespit edilmesi

B. ftgo-order-history Tablosu Tasarımı

DynamoDB depolama modeli, öğeler içeren tablolardan ve bir tablonun öğelerine muadil erişim yolları sağlayan indekslerden oluşur. Bir öğe, adlandırılmış niteliklerin bir koleksiyonudur. OrderHistoryDataAccess modülü, her Order'ı ftgo-order-history adlı DynamoDB tablosunda tek bir öğe olarak saklar.

Order sınıfının her alanı bir öğe niteliğine eşlenir:

- `orderCreationTime` ve `status` gibi basit alanlar tek değerli öğe niteliklerine eşlenir
- `lineItems` alanı, her satır için bir map olmak üzere map'lerin listesi olan bir niteliğe eşlenir

ftgo-order-history table

Primary key

orderId	consumerId	orderCreationTime	status	lineItems	...
...	xyz-abc	22939283232	CREATED	[{...}. {...}, ...]	...
...	...	...	...	....	...

Şekil 2. DynamoDB OrderHistory tablosunun ön yapısı

C. Secondary Index Tanımlanması

`findOrderHistory()` sorgu operasyonu, bir müşterinin siparişlerini artan yaşa göre sıralı olarak döndürür. Bu nedenle, `consumerId`'yi partition key ve `orderCreationDate`'i sort key olarak içeren bir primary key gerektirir.

Ancak `(consumerId, orderCreationDate)`'in ftgo-order-history tablosunun primary key'i olması mantıklı değildir çünkü benzersiz değildir. Çözüm, `findOrderHistory()`'nin ftgo-order-history tablosu üzerinde DynamoDB'nin secondary index olarak adlandırdığı şeyi sorgulamasıdır.

ftgo-order-history-by-consumer-id-and-creation-time global secondary index

Primary key

consumerId	orderCreationTime	orderId	status	...
xyz-abc	22939283232	cde-fgh	CREATED	...
...	...	...	...	...

ftgo-order-history table

Primary key

orderId	consumerId	orderCreationTime	status	lineItems	...
cde-fgh	xyz-abc	22939283232	CREATED	[{...}. {...}, ...]	...
...	...	...	...	....	...

Şekil 3. OrderHistory tablosu ve indeksinin tasarımı

Index, `ftgo-order-history-by-consumer-id-and-creati` olarak adlandırılır ve şu nitelikleri içerir:

- Primary key nitelikleri: `consumerId` ve `orderCreationTime`
- Non-key nitelikler: `orderId` ve `status`

D. Sorgu Implementasyonu

`findOrderHistory()` sorgu operasyonu, arama kriterlerini belirten bir filtre parametresine sahiptir. Bir filtre kriteri, döndürülecek siparişlerin azami yaşıdır. DynamoDB Query operasyonunun key condition expression özelliği sort key üzerinde aralık kısıtlamasını desteklediği için bu kolayca uygulanabilir.

Diğer filtre kriterleri non-key niteliklere karşılık gelir ve bir Boolean ifadesi olan filtre ifadesi kullanılarak uygulanabilir.

Örneğin, CANCELLED olan Order'ları bulmak için, OrderHistoryDaoDynamoDb `orderStatus =:orderStatus` sorgu ifadesini kullanır.

V. OrderHistoryDaoDynamoDb Sinifi

A. addOrder() Metodu

`addOrder()` metodu, ftgo-order-history tablosuna bir sipariş ekler. İki parametresi vardır: `order` ve `sourceEvent`. Order parametresi, OrderCreated olayından elde edilen eklenecek Order'dır.

```
public class OrderHistoryDaoDynamoDb {

  @Override
  public boolean addOrder(Order order,
                          Optional<SourceEvent> eventSource
                                                  ) {
   UpdateItemSpec spec = new UpdateItemSpec()
           .withPrimaryKey("orderId", order.getOrderId())
           .withUpdateExpression("SET orderStatus = :
                orderStatus, " +
                    "creationDate = :cd, consumerId = :
                         consumerId, " +
                    "lineItems = :lineItems, keywords = :
                         keywords, " +
                    "restaurantName = :restaurantName")
           .withValueMap(new Maps()
                    .add(":orderStatus", order.getStatus().
                         toString())
                   .add(":cd", order.getCreationDate().
                        getMillis())
                   .add(":consumerId", order.getConsumerId
                        ())
                   .add(":lineItems", mapLineItems(order.
                        getLineItems()))
                   .add(":keywords", mapKeywords(order))
                   .add(":restaurantName", order.
                        getRestaurantName())
                   .map())
           .withReturnValues(ReturnValue.NONE);
   return idempotentUpdate(spec, eventSource);
  }
}
```

Listing 2. addOrder() metodu bir Order ekler veya günceller

B. notePickedUp() Metodu

`notePickedUp()` metodu, DeliveryPickedUp olayı için olay işleyici tarafından çağrılır. Order öğesinin `deliveryStatus`'unu PICKED_UP olarak değiştirir.

```
@Override
public void notePickedUp(String orderId,
                         Optional<SourceEvent> eventSource)
                                                  {
 UpdateItemSpec spec = new UpdateItemSpec()
         .withPrimaryKey("orderId", orderId)
         .withUpdateExpression("SET #deliveryStatus = :
              deliveryStatus")
         .withNameMap(Collections.singletonMap("#
              deliveryStatus",
                 DELIVERY_STATUS_FIELD))
         .withValueMap(Collections.singletonMap(":
              deliveryStatus",
                 DeliveryStatus.PICKED_UP.toString()))
         .withReturnValues(ReturnValue.NONE);
 idempotentUpdate(spec, eventSource);
}
```

Listing 3. notePickedUp() metodu sipariş durumunu PICKED_UP olarak değiştirir

C. İdempotent Güncelleme

`idempotentUpdate()` metodu, mükerrer güncellemelere karşı koruyan bir koşul ifadesini UpdateItemSpec'e ekledikten sonra öğeyi günceller.

```
private boolean idempotentUpdate(UpdateItemSpec spec,
                                 Optional<SourceEvent>
                                      eventSource) {
 try {
  table.updateItem(eventSource.map(es ->
                   es.addDuplicateDetection(spec))
           .orElse(spec));
  return true;
 } catch (ConditionalCheckFailedException e) {
  // Hicbir sey yapma
  return false;
 }
}
```

Listing 4. idempotentUpdate() metodu mükerrer olayları yok sayar

SourceEvent sağlanmışsa, `idempotentUpdate()` metodu, daha önce açıklanan koşul ifadesini UpdateItemSpec'e eklemek için `SourceEvent.addDuplicateDetection()`'ı çağırır. Olay bir mükerrer ise `updateItem()` tarafından atılan `ConditionalCheckFailedException`'ı yakalar ve yok sayar.

D. findOrderHistory() Metodu

`findOrderHistory()` metodu, `ftgo-order-history-by-consumer-id-and-creation-tim` secondary index'ini kullanarak ftgo-order-history tablosunu sorgulayarak müşterinin siparişlerini alır.

```java
@Override
public OrderHistory findOrderHistory(String consumerId,
                                     OrderHistoryFilter
                                         filter) {

 QuerySpec spec = new QuerySpec()
         .withScanIndexForward(false)  // Artan yasa gore
             sirala
         .withHashKey("consumerId", consumerId)
         .withRangeKeyCondition(new RangeKeyCondition("
             creationDate")
                                  .gt(filter.getSince().
                                      getMillis()));

 filter.getStartKeyToken().ifPresent(token ->
       spec.withExclusiveStartKey(toStartingPrimaryKey(
           token)));

 Map<String, Object> valuesMap = new HashMap<>();

 String filterExpression = Expressions.and(
          keywordFilterExpression(valuesMap, filter.
              getKeywords()),
         statusFilterExpression(valuesMap, filter.getStatus
             ()));

 if (!valuesMap.isEmpty())
  spec.withValueMap(valuesMap);

 if (StringUtils.isNotBlank(filterExpression)) {
  spec.withFilterExpression(filterExpression);
 }

 filter.getPageSize().ifPresent(spec::withMaxResultSize);

 ItemCollection<QueryOutcome> result = index.query(spec);

 return new OrderHistory(
         StreamSupport.stream(result.spliterator(), false)
            .map(this::toOrder)
             .collect(toList()),
         Optional.ofNullable(result
               .getLastLowLevelResult()
               .getQueryResult().getLastEvaluatedKey())
            .map(this::toStartKeyToken));
}
```

Listing 5. findOrderHistory() metodu müşterinin eşleşen siparişlerini alır

VI. Sayfalandırma

Bazı müşterilerin çok sayıda siparişi olacaktır. Bu nedenle, `findOrderHistory()` sorgu operasyonunun sayfalandırma kullanması mantıklıdır. DynamoDB Query operasyonunun döndürülecek azami öğe sayısını belirten bir `pageSize` parametresi vardır.

Daha fazla öğe varsa, sorgunun sonucu null olmayan bir `LastEvaluatedKey` niteliğine sahiptir. DAO, `exclusiveStartKey` parametresini `LastEvaluatedKey`'e ayarlayarak sorguyu çağırarak sonraki öğe sayfasını alabilir.

DynamoDB pozisyon tabanlı sayfalandırmayı desteklemediği için, Order History Service istemcisine opak bir sayfalandırma token'ı döndürür. İstemci bu token'ı sonraki sonuç sayfasını istemek için kullanır.

VII. Mükerrer Olay Tespiti

Order History Service'in tüm olay işleyicileri idempotent'tir. Her biri Order öğesinin bir veya daha fazla niteliğini ayarlar. Ancak mükerrer olayları göz ardı etmenin zararı, Order öğesinin bazen geçici olarak güncel olmaması olabilir.

Mükerrer bir olay alan olay işleyici, Order öğesinin niteliklerini önceki değerlere ayarlayacağından verilerin güncel olmaması söz konusu olabilir. Order öğesi, daha sonraki olaylar yeniden teslim edilene kadar doğru değerlere sahip olmayacaktır.

Daha önce açıklandığı gibi, verilerin güncel olmamasını önlemenin bir yolu mükerrer olayları tespit edip atmaktır. OrderHistoryDaoDynamoDb, her öğede ona güncellenmesine neden olan olayları kaydederek mükerrer olayları tespit edebilir.

OrderHistoryDaoDynamoDb DAO, her aggregate instance'dan alınan olayları, değeri alınan en yüksek olay kimliği olan «`aggregateType`»«`aggregateId`» adlı bir nitelik kullanarak izleyebilir. Bir olay, nitelik mevcutsa ve değeri olay kimliğinden küçük veya ona eşitse mükerrerdir.

VIII. Sonuç ve Değerlendirme

CQRS view uygulanırken ele alınması gereken çok sayıda konu bulunmaktadır:

- Veritabanı seçimi
- Güncellemeleri ve sorguları verimli bir şekilde uygulayan veri modelinin tasarlanması
- Eşzamanlı güncellemelerin işlenmesi
- Mükerrer olaylarla başa çıkma

Kodun karmaşık olan tek kısmı DAO'dur, çünkü eşzamanlılığı düzgün bir şekilde ele almalı ve güncellemelerin idempotent olmasını sağlamalıdır.

DynamoDB gibi NoSQL veritabanları, güçlü ölçeklenebilirlik ve esneklik sağlarken, dikkatli veri modelleme ve sorgu tasarımı gerektirir. Secondary index'ler ve conditional update'ler gibi DynamoDB'nin özellikleri, güvenilir ve hızlı CQRS view'ları oluşturmak için etkili bir şekilde kullanılabilir.